

Aprendizaje No Supervisado

Clustering · K-Means · PCA

Modulo II · Ciencia de Datos e Introducción a Machine Learning

Contenidos de la unidad

01

Supervisado vs. No supervisado

Diferencias clave y cuándo aplicar cada enfoque

02

Algoritmo K-Means

Funcionamiento, inercia y limitaciones

03

Elbow Method y Silhouette Score

Cómo elegir el número óptimo de clusters

04

Escalado de datos

Por qué es crítico antes de aplicar K-Means

05

PCA para visualización

Reducción de dimensionalidad y visualización 2D

Supervisado vs. No Supervisado

Diferencias clave entre ambos paradigmas de Machine Learning



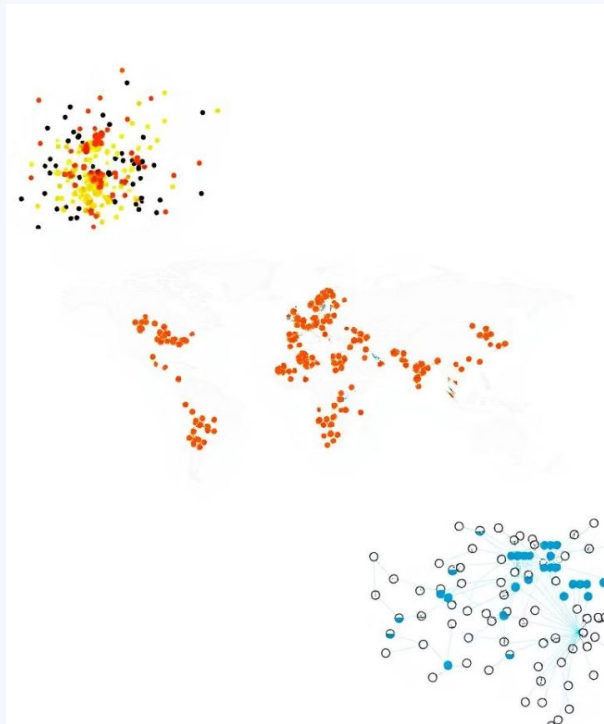
¿Cuál es la diferencia?

Aprendizaje Supervisado	
Etiquetas	Requiere etiquetas (y)
Objetivo	Predecir salida conocida
Evaluación	Accuracy, MAE, F1, RMSE
Ejemplos	Regresión, clasificación
Casos de uso	Precios, diagnóstico médico

VS

Aprendizaje No Supervisado	
Etiquetas	No requiere etiquetas
Objetivo	Descubrir estructuras ocultas
Evaluación	Silhouette, Inercia, Inertia
Ejemplos	Clustering, PCA, Autoencoders
Casos de uso	Segmentación, detección anomalías

Aplicaciones de K-Means Clustering



Marketing

Segmentación de clientes para campañas personalizadas.

Biología

Clasificación de especies o genes según características.

Salud

Detección de enfermedades a partir de datos clínicos.

Redes sociales

Análisis de comunidades y patrones de interacción.

El clustering se utiliza en múltiples campos para descubrir patrones. Esto ayuda a comprender mejor los datos y tomar decisiones informadas.

Algoritmo K-Means

El algoritmo de clustering más popular en Machine Learning



¿Qué hace K-Means?

K-Means divide n observaciones en k grupos (clusters) de forma que cada punto pertenezca al cluster cuyo centroide (media) sea el más cercano. El valor k se define antes de ejecutar el algoritmo.

Pasos del algoritmo

1

Inicializar centroides

Seleccionar k puntos aleatorios (o K-Means++) como centroides iniciales

3

Recalcular centroides

El centroide de cada cluster pasa a ser la media de sus puntos asignados

2

Asignar puntos

Cada punto se asigna al centroide más cercano (distancia euclidiana)

4

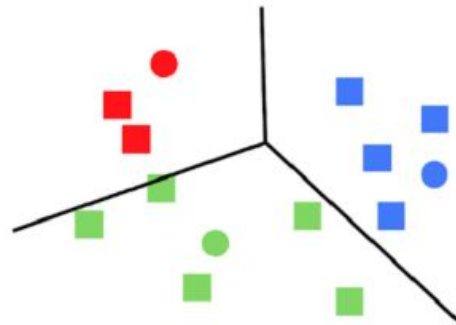
Iterar hasta converger

Repetir pasos 2-3 hasta que los centroides no cambien significativamente

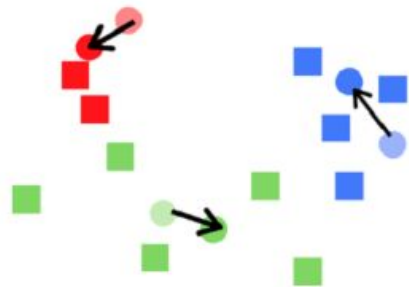
Step 1



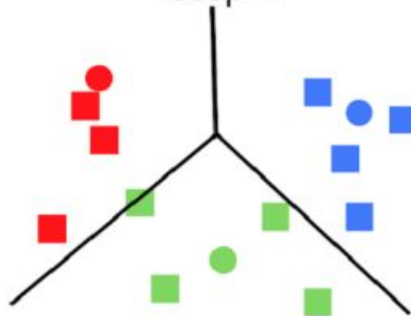
Step 2



Step 3



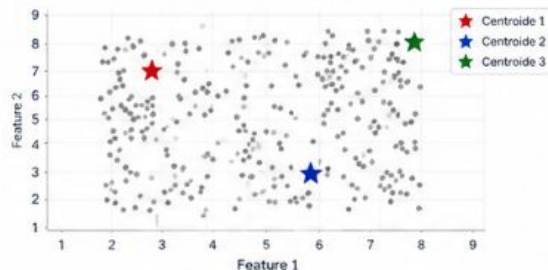
Step 4



Visualización Iterativa de K-Means

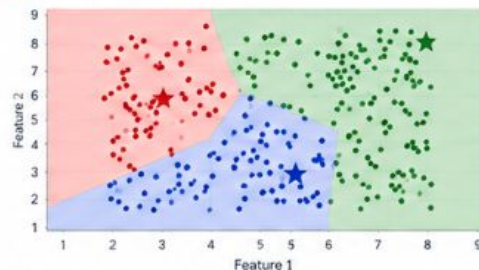
K-Means agrupa los datos en K clústeres **minimizando la varianza intra-clúster**.

1 Inicialización



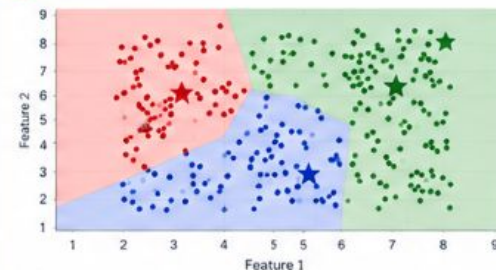
Se eligen **K centroides iniciales** de forma aleatoria (K = 3 en este ejemplo).

2 Asignación

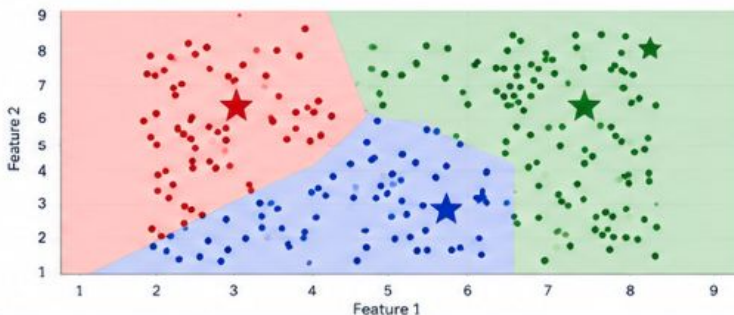


Cada dato se asigna al centroide más cercano utilizando la **distancia euclidiana**.

3 Actualización



Los centroides se recalculan como el **promedio** de los puntos asignados a cada clúster.



Convergencia y Resultado Final

Los pasos de asignación y actualización se repiten iterativamente hasta que los centroides ya no cambian de forma significativa (**convergencia**). En ese punto, los clústeres son estables.



Iteración

Asignar → Actualizar
→ Repetir



Objetivo

Minimizar la inercia total (WCSS)



Convergencia

Los centroides dejan de cambiar significativamente.

La **inercia total (WCSS: Within-Cluster Sum of Squares)** mide la suma de las distancias cuadradas de cada punto a su centroide. El algoritmo K-Means busca minimizar este valor para obtener clústeres compactos y bien definidos.

Implementación en Python

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Escalar los datos (OBLIGATORIO)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. Entrenar K-Means con k=3
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10, # múltiples inicializaciones
    init='k-means++'
)
kmeans.fit(X_scaled)

# 3. Resultados
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
inertia = kmeans.inertia_

print(f'Inercia (WCSS): {inertia:.2f}')
```

Atributos del modelo

`labels_`

Etiqueta de cluster (0..k-1) para cada punto

`cluster_centers_`

Coordenadas de los k centroides (en espacio escalado)

`inertia_`

WCSS: suma de distancias cuadradas al centroide.
Menor = mejor

`n_init=10`

⚠ K-Means asume clusters esféricos y de tamaño similar.
Escalar siempre antes de aplicar.

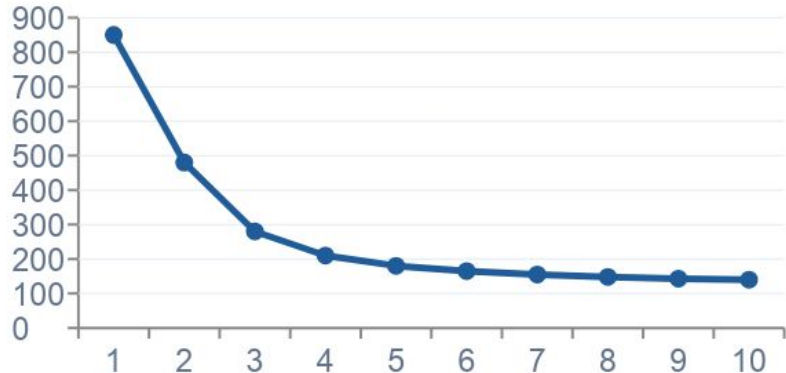
¿Cuántos Clusters necesito?

Elbow Method y Silhouette Score para elegir k óptimo

Método del Codo (Elbow Method)

Entrena K-Means para distintos valores de k y grafica la inercia (WCSS). La curva cae rápidamente al principio y luego se aplana. El punto de inflexión (codo) indica el k óptimo.

Gráfica del Método del Codo



```
inercias = []
k_range = range(1, 11)

for k in k_range:
    km = KMeans(n_clusters=k,
                random_state=42, n_init=10)
    km.fit(X_scaled)
    inercias.append(km.inertia_)

plt.plot(k_range, inercias, 'bo-')
plt.xlabel('k')
plt.ylabel('Inercia')
plt.title('Metodo del Codo')
plt.show()
```

Limitación del Elbow Method

El "codo" no siempre es obvio en la gráfica. En esos casos, complementar con el Silhouette Score para una decisión más objetiva.

Silhouette Score

$$s(i) = [b(i) - a(i)] / \max(a(i), b(i))$$

a(i)

Distancia media del punto i a los demás puntos de su mismo cluster (cohesión interna). Debe ser pequeña.

b(i)

Distancia media del punto i al cluster vecino más cercano (separación externa). Debe ser grande.

s(i)

Score de cada punto. Promedio de todos = Silhouette Score global del clustering.

Interpretación

Cercano a +1

Clusters bien definidos y separados

Cercano a 0

Punto en frontera entre clusters

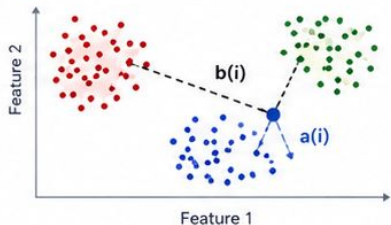
Negativo

Punto mal asignado a su cluster

Silhouette Score: ¿Qué tan buenos son nuestros clústeres?

Mide qué tan bien cada punto está asignado a su clúster comparando **cohesión** y **separación**.

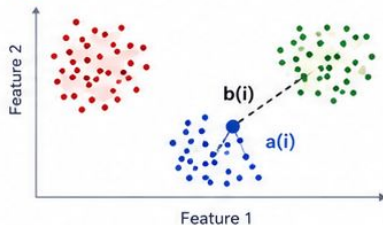
1 Silhouette alto (cercano a 1)



- ✓ Muy cerca de su propio clúster (a pequeño) y lejos del más cercano (b grande).

$s(i) \approx 0.7 \text{ a } 1 \rightarrow$ bien asignado

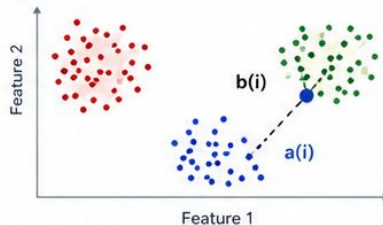
2 Silhouette cercano a 0



- ⚠ Está cerca de la frontera entre clústeres (a y b similares).

$s(i) \approx -0.1 \text{ a } 0.1 \rightarrow$ en la frontera

3 Silhouette negativo (cercano a -1)



- ✗ Más cerca de otro clúster que del propio (a grande y b pequeño).

$s(i) < 0 \rightarrow$ posible mala asignación

¿Cómo se calcula?

Para cada punto i :

- $a(i)$: distancia promedio del punto i a todos los puntos de su mismo clúster.
- $b(i)$: distancia promedio del punto i a todos los puntos del clúster vecino más cercano.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

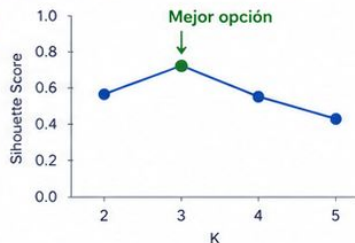
El Silhouette Score global es el promedio de $s(i)$ de todos los puntos.

$$-1 \leq s(i) \leq 1$$

¿Para qué nos sirve con K-Means?

Lo usamos para elegir el número de clústeres (K). Probamos distintos K y nos quedamos con el que obtiene el mayor Silhouette Score promedio.

K	Silhouette Score
2	0.52
3	0.71
4	0.58
5	0.46



💡 Elegimos el K que maximiza el Silhouette Score.
En este ejemplo:
K = 3

Guía rápida de interpretación

0.7 a 1.0



Clústeres compactos y bien separados. Los puntos están bien asignados.

0.25 a 0.7



Calidad aceptable. Hay cierta superposición entre clústeres.

-1.0 a 0.25



Clústeres mal definidos. Muchos puntos pueden estar mal asignados.



El Silhouette Score complementa (no reemplaza) la inspección visual y el conocimiento del problema.

¿Por qué escalar antes de K-Means?

✗ Sin escalar

Ingreso: 20,000–100,000 | Edad: 18–80

K-Means usará casi solo el ingreso → clusters incorrectos

✓ Con StandardScaler

Ingreso: -1.2 a 1.8 | Edad: -1.5 a 2.0

Ambas variables contribuyen equitativamente → clusters correctos

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler() # media=0, std=1
X_scaled = scaler.fit_transform(X) # todo el dataset:
no hay y
```

Checklist antes de hacer clustering

- Eliminar o imputar valores nulos
- Codificar variables categóricas
- Escalar todas las variables numéricas
- Detectar y tratar outliers extremos
- Seleccionar features relevantes

Regla de oro: SIEMPRE escalar ANTES de K-Means. Acá no hay variable y: a diferencia del supervisado, no hace falta separar train/test — el scaler se ajusta sobre todo el dataset.

```
# En producción: fit() una sola vez con datos históricos; a clientes nuevos solo transform() (evita leakage)
```

Limitaciones de K-Means

Sensibilidad y Preparación

- **Sensible a la escala** de las variables (requiere escalado previo).
- Los **outliers** pueden desplazar los centroides drásticamente.
- No trabaja naturalmente con **variables categóricas**.

Asignación e Interpretación

- **Asignación forzada**: asocia cada punto a un cluster, aunque sea un atípico extremo.
- Los clusters encontrados **no siempre tienen significado** o relevancia real en el negocio.

Supuestos y Estructura

- Hay que **definir K previamente**, sin conocer la cantidad real.
- Funciona mejor con clusters **compactos y redondeados**.
- El resultado varía según la **inicialización** de centroides.

Idea Clave

K-Means propone una forma estructurada de organizar los datos bajo ciertos supuestos matemáticos; **no descubre automáticamente una "verdad"** absoluta u oculta sobre ellos.

Consejo: Si K-Means falla por sus supuestos, evalúe alternativas como DBSCAN o GMM

PCA para visualización de clusters

Reducción de dimensionalidad y visualización de datos de alta dimensión



¿Qué es PCA y por qué usarlo?

Problema

Con 10+ variables no podemos visualizar los clusters directamente en 2D o 3D.

Solución

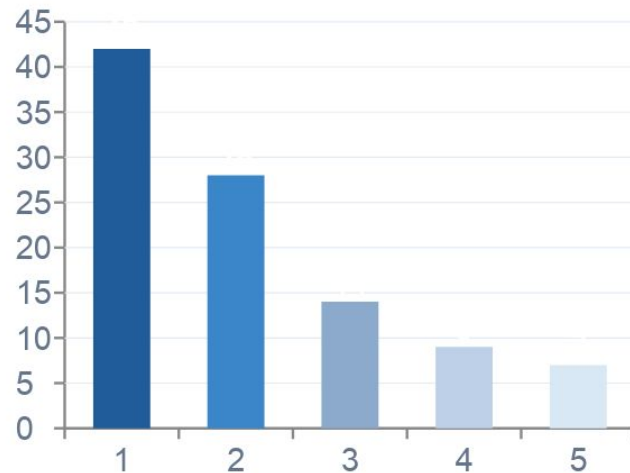
PCA proyecta los datos en nuevas variables (componentes principales) que maximizan la varianza explicada.

Resultado

Con 2 componentes (PC1 y PC2) podemos graficar todos los puntos y ver si los clusters están bien separados.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled) # siempre sobre datos
escalados
print(pca.explained_variance_ratio_) # % varianza por
componente
```

Varianza explicada acumulada



Regla práctica: elegir el número de componentes que expliquen $\geq 95\%$ de la varianza total.